

Self-Evaluation Sheet

Project: Meal Planning and Recipe Management Web Application

1. Overview

This section provides a critical evaluation of the project, reflecting on the strengths, weaknesses, and overall learning outcomes achieved during development. The evaluation is aligned with the module's marking criteria, including technical implementation, design, functionality, and overall quality.

2. Evaluation Process Flow



Figure 1: Self-Evaluation Process

Flow Explanation

The evaluation process follows a continuous improvement cycle:

- Analyse completed work
- Identify strengths and weaknesses
- Reflect on learning outcomes
- Propose improvements

3. Strengths

The project demonstrates several key strengths:

3.1 Strong Full-Stack Integration

The application successfully integrates Angular (frontend), Flask (backend), and MongoDB (database), demonstrating a clear understanding of full-stack architecture.

3.2 Advanced Feature Implementation

Features such as:

- Nutrition tracking using BMR calculations
 - Dynamic recipe filtering
 - Role-based access control
- showcase the ability to implement beyond basic requirements.

3.3 Clean and Modular Architecture

The use of Angular's component-based structure improves maintainability and scalability. Services are used effectively to handle API communication.

3.4 User Interface Design

The application includes:

- Dashboard-style layout
- Card-based design

- Responsive UI
- Interactive elements (hover effects, animations)

This enhances user experience and usability.

4. Weaknesses

Despite its strengths, the project has some limitations:

4.1 Limited Automated Testing

Testing was conducted manually, and no automated testing frameworks (e.g., unit tests) were implemented.

4.2 Authentication Limitations

Although Auth0 was integrated, backend-level authentication validation could be further strengthened using JWT or session-based verification.

4.3 Performance Considerations

The application has not been tested under high-load conditions, which may affect scalability in real-world deployment.

5. Learning Outcomes

This project significantly enhanced the following skills:

- Full-stack development using Angular, Flask, and MongoDB
- Designing and consuming RESTful APIs
- Implementing responsive UI/UX principles
- Managing application state and asynchronous data
- Debugging and problem-solving in a real-world scenario

6. Reflection

The development process involved overcoming multiple challenges, including API integration, state management, and UI design. Through iterative development and testing, the application evolved into a functional and structured system.

This project provided valuable insight into real-world software development practices, particularly in building scalable and maintainable applications.

7. Future Improvements

If further development time were available, the following improvements would be implemented:

- Integration of automated testing frameworks
- Enhanced authentication using JWT
- Deployment to a cloud platform
- Implementation of advanced data visualisation tools
- Addition of real-time features

8. Justification Against Marking Criteria

The project meets high-level criteria in the following areas:

- **Technical Implementation:** Strong use of Angular, Flask, and MongoDB
- **Functionality:** All core features implemented successfully
- **Design:** Clean, responsive, and user-friendly interface
- **Complexity:** Includes advanced features such as nutrition tracking and role-based access